EdgeFault-Bench: A Formally Specified and Extensible Software Framework for Auditable Industrial Fault-Diagnosis Experiments

ANONYMOUS AUTHOR(S)

Cross-condition fault-diagnosis experiments are unusually exposed to experiment-integrity failures: windows cut from one recording leak across train/test partitions, held-out conditions silently participate in preprocessing and model selection, seeds disappear from reported results, and provenance is unrecoverable. Existing fault-diagnosis benchmarks standardize *what* is evaluated, and experiment trackers record *whatever is logged*, but neither enforces the validity of the experiment itself. We treat the benchmark as a software system under formal specification. EdgeFault-Bench specifies the experiment lifecycle as a guarded state machine with six integrity invariants (source integrity, partition exclusivity, test isolation, seed completeness, provenance completeness, plugin contract), enforces them at the kernel boundary of an extensible plugin architecture, and verifies the design at three levels: exhaustive model checking of the abstract lifecycle (46,686 reachable states, no invariant violations), property-based testing with independently recomputed oracles, and a twelve-entry fault-injection catalogue in which every planted integrity violation is rejected fail-closed. The framework carries six dataset integrations—including a gearbox and a multimodal acoustic/vibration pair integrated without dataset-specific kernel code—versioned combined-condition and leave-one-device-out task manifests, group-bootstrap confidence intervals over recording-level resampling units, and result bundles whose input artefacts are content-hashed and re-verifiable. Framework overhead is measured (15–30× bare JSON parsing for load-and-validate; sub-millisecond result validation), and cross-Python and cross-OS executions reproduce metrics exactly. We report the design, the verification stack, and honest boundaries: the formal model covers the abstract guard logic, not the Python implementation, and third-party adoption evidence is not yet available.

Additional Key Words and Phrases: experiment integrity; formal specification; model checking; benchmark frameworks; fault diagnosis; reproducibility; plugin architectures

1 INTRODUCTION

Industrial fault-diagnosis research increasingly claims generalization: models are praised for surviving *operating-condition shift*—a bearing trained at one load or on one test rig and evaluated on another. The claim is only as good as the experiment that produces it, and those experiments are fragile in specific, recurring ways. Vibration records are typically cut into overlapping windows; unless windows of one recording are kept in a single partition, near-duplicate samples straddle the train/test boundary and inflate scores. Normalization statistics fitted over the whole dataset quietly fold held-out conditions into training. Model selection peeks at the test condition. A seed that crashed is dropped, leaving three reported runs of an experiment registered with four. And when a result is challenged, the chain from numbers back to data files, code, and configuration no longer exists. Recent standardization work in the domain makes exactly this complaint: ad hoc partitions and evaluation setups make cross-condition results incomparable across papers [1, 17].

A natural first reaction is to standardize the benchmark: fix the datasets, the splits, and the metrics, as domain-generalization research did for vision (DomainBed [3]) and as fault-diagnosis benchmark studies have begun to do [16, 17]. This standardizes *what* is evaluated. It does not, however, guarantee *that the evaluation is valid*: leakage controls, test-set isolation, seed completeness, and provenance remain conventions living in scripts, to be re-implemented—and re-broken—by every extension. The complementary reaction is infrastructure: experiment trackers such as MLflow [12] record parameters, code versions, and artefacts. But trackers are record-keepers; they enforce nothing about partition exclusivity or seed completeness, and integrity remains voluntary and after the fact.

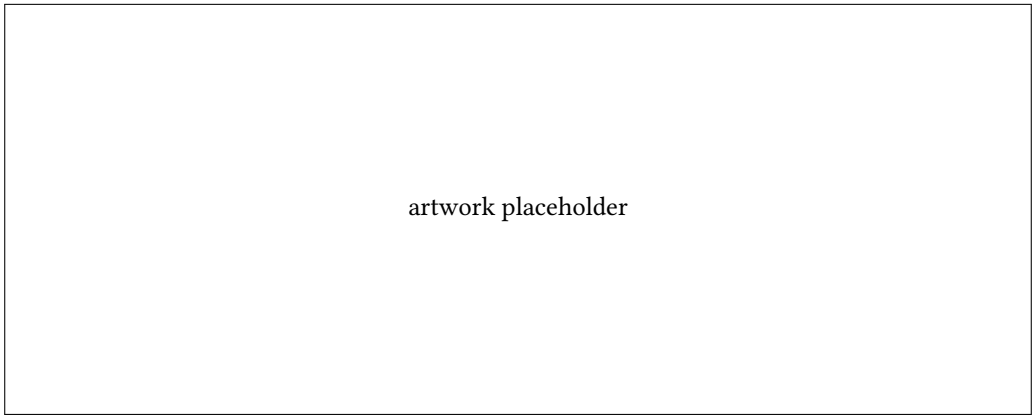


Fig. 1. The same experiment under two regimes, measured on the real bearing archive with identical model, policy, and seeds (audit policy; artefact: `results/verification/leakage-motivated-example-2026-11-07.json`). Left: a conventional window-level random split lets windows of one recording straddle the train/test boundary (all 36 recordings shared across partitions in every seed) and inflates mean test Macro-F1 to 0.449 against 0.293 under a recording-grouped split with zero shared recordings. Right: rather than executing such an experiment, EdgeFault-Bench rejects the identical artefact at the TaskAudited transition with a machine-readable invariant violation (*recording R spans partitions train/test*, INV-2); nothing partial survives.

This paper takes a third position: *the benchmark itself should be a software system under formal specification*, in which experiment integrity is not a convention but a property enforced by architecture and checked by tools. Concretely, we ask whether the validity of an industrial ML experiment can be guaranteed by a small set of kernel-enforced invariants, whether third-party datasets and models can be integrated without any ability to violate those invariants, and whether the guarantee itself can be verified rather than asserted.

Making that position real raises three challenges. First, *heterogeneity against integrity*: real datasets span file formats, sampling rates (sometimes unstated by the source), sensor modalities, and condition taxonomies, while model code spans classical estimators and deep frameworks; every extension point is an opportunity to break an invariant. Second, *enforcement must be verifiable, not declared*: invariants that live only in documentation are indistinguishable from absent ones; the guards must be explicit, machine-checkable, and demonstrably effective against injected violations. Third, *validity must survive scale and distribution*: hashing, streaming, and result bundles must work on multi-gigabyte archives, and the framework's own cost must be measured and acceptable.

EdgeFault-Bench answers these with a four-module design. A *data-and-protocol* module fixes canonical recording and window identity and expresses tasks—including combined-condition and leave-one-device-out designs—as versioned manifests rather than code. An *extensible execution* module streams signals from checksum-verified sources through dataset, preprocessing, and model plugins; training plugins receive partition-restricted views and are architecturally unable to read the test partition, which is materialized only inside the evaluation step. A *formal integrity* module specifies the experiment lifecycle as a guarded state machine over six invariants, checked exhaustively with TLA$^+$/TLC [7] at the design level and continuously by property-based tests [9] and a fault-injection catalogue at the implementation level. A *reproducibility* module emits result bundles whose inputs are content-hashed, whose seeds biject with the registered task seeds, and whose group-bootstrap confidence intervals resample recordings rather than windows.

This article contributes:

(1) a formalized specification of reproducible industrial ML experiments: an eight-state lifecycle with guarded transitions and six integrity invariants (§3);
(2) an end-to-end extensible architecture that enforces the specification at its kernel boundary for heterogeneous datasets and both classical and deep model backends (§4, §5);
(3) an automated integrity-verification and provenance stack—model checking, oracle-based property tests, a twelve-entry fault-injection catalogue, and hash-recheckable result bundles (§5, §6);
(4) an empirical evaluation across heterogeneous datasets (bearings, a gearbox, acoustic and vibration modalities), independent plugin packaging, and execution environments, including measurements of the framework's own overhead (§6).

We are deliberate about two boundaries. The formal model verifies the *abstract guard logic* of the design, not the Python implementation; the implementation is covered by the runtime checks, property tests, and mutation results, and we say so wherever the distinction matters. And the framework has, so far, one author; evidence from third-party integrators is being solicited but is not part of this article's claims.

2 RELATED WORK

Fault-diagnosis benchmarks and standardization. The closest domain work is the application-oriented benchmark of Zhao, Zio, and Shen [17], which unifies public rotating-machinery datasets under a common domain-generalization protocol with open code (`DG_benchmark` [16]), and a 2025 standardization framework in IEEE Access that formalizes fault-level and operating-condition factors for controlled evaluation [1]. Both respond to documented comparability failures of ad hoc setups, and both standardize the *content* of evaluation: datasets, shift factors, partitions, metrics. Neither prescribes, enforces, or verifies the *validity mechanisms* of the experiments built on top—recording-level leakage control, test isolation, seed completeness, or provenance chains. That layer is the subject of this paper, and the two lines are complementary: the protocols those works define could be expressed as EdgeFault-Bench task manifests and would then inherit the kernel's enforcement.

Domain-generalization evaluation practice. DomainBed [3] established rigorous model-selection discipline for domain generalization in vision, showing that fair selection changes algorithm rankings. Its concern—that evaluation protocol, not algorithm, often drives conclusions—is the same concern we transport into industrial experimentation; DomainBed, however, distributes datasets and training scripts rather than installable integrity contracts, and its machine-readable artefacts do not model machinery-specific notions such as the recording as the unit of independence. General time-series ecosystems such as aeon [11] provide algorithm breadth; EdgeFault-Bench deliberately reuses such ecosystems for estimators where possible and concentrates on what they do not provide: condition-held-out task semantics, source-verified streaming, and audit artefacts.

Experiment infrastructure and reproducibility. MLflow [12] and similar trackers log whatever an experiment emits; they neither define partitions nor reject a bundle with a missing seed. Recent reproducibility analyses argue that releasing code and dependency lists alone does not make ML benchmarks runnable or trustworthy [8], and infrastructure benchmarking has begun treating the experimental platform itself as the object of study [6]; benchmark suites for autonomous research systems evaluate whether agents can replicate results [2]. These works share our diagnosis that the platform layer decides reproducibility. Our contribution to that conversation is a platform whose

Table 1. Lifecycle transitions, guards, and enforcing mechanisms.

Transition	Guard (invariant)	Runtime mechanism
t_1 acquire	INV-1	checksum-verified download; re-verification at every stream open
t_2 audit	INV-2 (recordings)	fail-closed recording audit
t_3 prepare	INV-2 (windows), INV-3	window-inheritance check; kernel-restricted fit scope
t_4 construct	INV-6	plugin registry and contract validation
t_5 train	INV-3, INV-4	partition-restricted data views; versioned training policy
t_6 evaluate	INV-3, INV-4	test view materialized only here, once per seed
t_7 validate	INV-4, INV-5	seed-bijection and provenance validation; input-hash recompute
t_8 publish	INV-1, INV-5	immutable release with checksums

integrity properties are *specified formally and attacked deliberately* rather than asserted, with the fault-injection results to show for it.

Formal methods in ML tooling. TLA$^+$ and TLC are established tools for specifying and model-checking concurrent and distributed systems [7]; property-based testing with Hypothesis [9] is an established implementation-level verification technique. To our knowledge, applying this stack—a guarded lifecycle state machine, model-checked invariants, oracle-based property tests, and a mutation catalogue—to the integrity of ML *benchmark infrastructure* has not been done before; adjacent formal-methods work targets training systems and distributed learning, not the experiment lifecycle itself. A dedicated literature pass before submission will re-examine this positioning.

3 REQUIREMENTS AND FORMAL SPECIFICATION

3.1 Users and requirements

Three user roles drive the requirements. *Model researchers* add and compare estimators without rebuilding dataset parsing, splits, metrics, or provenance capture. *Dataset maintainers* publish adapters for their archives with source identity, licence, checksums, and recording boundaries intact. *Reviewers and reproducibility researchers* must be able to verify—without trusting the authors—that recordings do not cross partitions, that held-out conditions did not select models, that every registered seed is accounted for, and that results map to specific data, code, and environment artefacts. All three roles interact with the framework only through its public contracts; the requirements therefore reduce to one sentence: *extensions must be expressive enough for heterogeneous datasets and models, yet structurally incapable of violating experiment integrity.*

3.2 Experiment lifecycle

We model one (task, model-plugin) experiment as a guarded state machine over the states Empty → SourcesVerified → TaskAudited → DataPrepared → ModelConstructed → Trained$_s$ → Evaluated$_s$ → ResultValidated → Published, plus a terminal Rejected state. Each transition $t_1 - t_8$ carries a guard; a guard that fails sends the experiment to Rejected with a machine-readable reason, and nothing partial survives: the discipline is fail-closed. Table 1 lists the transitions with their guards and the runtime mechanism that enforces each.

3.3 Integrity invariants

Six invariants hold in every committed state:

INV-1 (Source integrity). Every data file matches its registered byte size and SHA-256, and every result references its input artefacts by content hash.

INV-2 (Partition exclusivity). The partition assignment is a total function over recordings; every window inherits its recording's partition; no recording or window appears in two partitions.

INV-3 (Test isolation). Test data is read only in t_6; preprocessing statistics are fitted on train/validation only; model selection and early stopping use validation only. Enforced structurally: training plugins never receive test rows.

INV-4 (Seed completeness). Result runs biject with the task's registered seeds, in the frozen order; failures are retained as explicit placeholders.

INV-5 (Provenance completeness). A valid result names its task, dataset, plugin, code commit, environment, and training policy, and its input hashes recompute.

INV-6 (Plugin contract). External plugins satisfy interface, identity, and version constraints; collisions and mismatches fail closed.

3.4 Model checking

The lifecycle and invariants are written in TLA$^+$ (`formal/ExperimentLifecycle.tla`). Bad candidates—tampered partitions, windows claiming a foreign partition, fits that attempt test data, contract-less plugins, incomplete bundles—are explored non-deterministically, so every guard's rejection path is exercised, not assumed. TLC checks nine properties (type safety, the six invariants, and the fail-closed discipline) over a small finite model (three recordings, three windows, two seeds). The result: *model checking completed with no errors*—89,476 states generated, 46,686 distinct states, search depth 11, no deadlocks. Two modelling defects were found and fixed by TLC during development (an invariant scoped to a state where its variables are not yet meaningful, and an underspecified rejecting branch), which is evidence the check has teeth. The archived log and a reproduction script ship with the repository, and CI re-runs the check on every push.

Scope of the formal claim. This is bounded model checking of an *abstract* lifecycle: it verifies the guard logic of the design. It is not a proof about the Python implementation. Implementation-level coverage comes from the runtime contract checks (§5), the property-based tests, and the fault-injection catalogue (§6); we keep these layers explicit rather than letting the phrase "formally specified" absorb them.

4 SYSTEM ARCHITECTURE

The kernel is organized in four modules, each owning one concern of the lifecycle; the formal guards of §3 sit at the module boundaries.

4.1 Data and protocol management

The unit of independence is the *recording*: one independently acquired sensor file with a stable identifier, label, condition domains (load, device, fault family, severity, …), sampling rate where the source states one, and a source hash. Windows cut from a recording carry (*window_id*, *recording_id*, *start_sample*, *end_s* which is what makes INV-2 checkable at window granularity: a window claiming a foreign partition is detectable both within a table and against the adapter's canonical metadata.

Tasks are data, not code. A task manifest names the held-out condition field(s), the partition assignment, labels, window rule, and the frozen seed list. Two partition modes exist: single-field

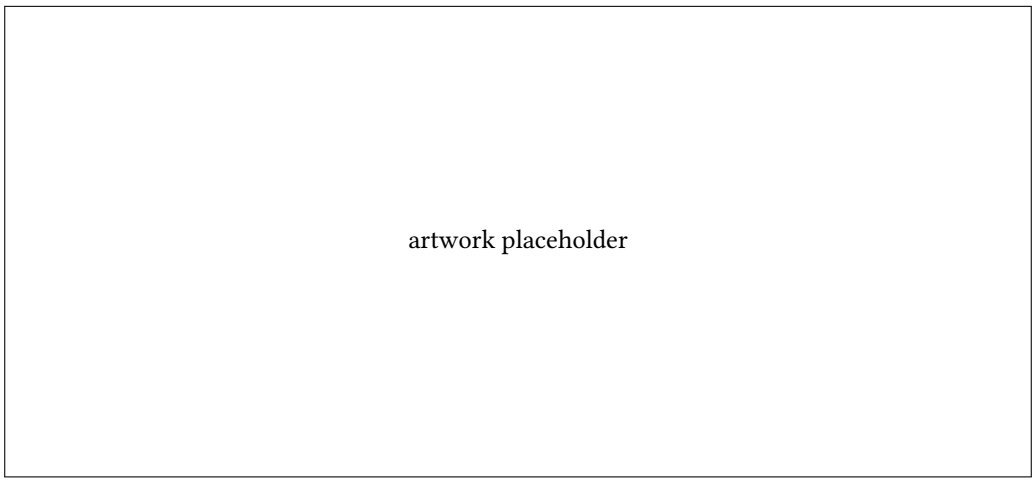


Fig. 2. Architecture overview. A recording-identity data layer feeds the extensible execution module, whose plugins (dataset signal streams, fit-scope-restricted preprocessing, policy-bound models) never receive the test partition: the locked channel opens only inside the evaluation transition t_6. The guarded lifecycle (states EMPTY to PUBLISHED, with REJECTED) sits above the modules and pins each invariant INV-1–INV-6 to the transition that enforces it; the reproducibility module emits result bundles whose inputs are content-hashed.

hold-out (one domain field's values split across train/validation/test), and *combined-condition* hold-out, where partitions are assigned over tuples of several condition fields—for example (load, defect size) pairs—so that a task can hold out a *combination* never seen in training. Leave-one-device-out families are simply sets of single-field manifests in which each device takes a turn as the unseen test device.

Honest metadata is a design rule: when a dataset's source does not state a sampling rate, the adapter records `None`; consumers that need physical frequencies (spectral features) then fail closed instead of assuming a value.

4.2 Extensible execution

Dataset, preprocessing, and model plugins are discovered through standard Python entry points and validated fail-closed (identity match, method presence, collision rejection). The dataset contract separates a metadata-only tier from a *signal capability*: an adapter that can `open` a recording returns a stream with channel layout, declared rate (possibly `None`), and total length, re-verifying the source checksum at open time. Window extraction streams each registered recording through this interface—kernel code never parses a dataset-specific format.

Test isolation is architectural. The kernel constructs partition-restricted data views; an estimator's `fit` receives train and validation views only, and the test view is materialized exclusively inside the evaluation step. A plugin cannot ask for test data because no API path carries it. Training itself is governed by a kernel-owned, versioned *training policy* manifest (optimizer, learning rate, budget, early stopping on validation Macro-F1, determinism flags): scikit-learn [15] and PyTorch [14] backends execute under the same policy object through one command, so backend choice cannot smuggle in a different model-selection regime.

4.3 Formal integrity layer

The TLA$^+$ specification of §3 is the reference for the runtime guards: each transition of Table 1 names the invariant it enforces and the code that enforces it, and that mapping is kept in the repository as part of the specification document. The runtime layer consists of the recording audit (identity, domain coverage, partition totality and exclusivity), the window inheritance check, the fit-scope guard, the plugin validators, and the result validator; each fails closed with a structured, machine-readable error.

4.4 Reproducibility and evaluation

Result bundles are versioned JSON documents whose inputs are content-hashed (task manifest, prepared table or dataset manifest, training policy), whose runs biject with registered seeds including explicit failures, and which retain per-sample test predictions (truth, prediction, evaluation group, recording id) for downstream statistics. Because windows of one recording are not independent samples, the statistics layer reports window-level scores alongside group-bootstrap percentile intervals that resample whole recordings, and compares two models with paired per-seed group-level bootstrap. Storage is layered: JSON as the auditable interchange, optional Parquet for large feature matrices, raw signals never copied but streamed from checksum-verified sources.

5 IMPLEMENTATION

EdgeFault-Bench is a Python package (v1.2.0) with a single `edgefault` CLI and an extensive audit-trail culture inherited from its research-software phase: every design decision is recorded in numbered decision records, every dataset in a data card, and every executed workflow in dated audit documents. The test suite currently stands at 198 passing tests, including the verification artefacts described below.

Contracts and plugins. Immutable dataclasses validate at construction (partition disjointness, domain coverage, seed uniqueness, window-identity completeness). Dataset plugins register through the `edgefault_bench.datasets` entry-point group; model plugins declare a backend (`sklearn` or `pytorch`) and are built per seed with class count and channel count from the kernel; preprocessing plugins register separately and declare whether they emit windows or feature vectors. Built-in adapters are registered exactly like external ones—there is no privileged internal path—and a test asserts that no lifecycle kernel module contains any dataset identifier, with the two known debts (the fetch dispatcher and a frozen demonstration asset) listed explicitly.

Versioned schemas. Task, prepared-table, and result documents are governed by versioned JSON Schemas; validation dispatches on `schema_version`, so v1 artefacts remain valid and v2 artefacts are never forced through their predecessors. Prepared tables v2 require window identity on every row; result bundles v2 add hashed inputs, per-seed status, retained predictions, and framework overhead timings.

Verification stack. Beyond the model-checked specification, the repository carries three implementation-level verification layers. *Property-based tests* (Hypothesis) pair generated artefacts—valid and invalid task partitions, recording sets with planted duplicates, unassigned conditions, or foreign labels; prepared tables with duplicate or partial window identity; bundles with dropped, duplicated, or reordered seeds—with independently recomputed accept/reject oracles, so agreement is checked, not just acceptance. *Fault injection* packages twelve integrity violations (§6) behind one command, `edgefault mutation audit`, which renders the detection matrix and reports honestly: a test deliberately sabotages one mechanism and requires the matrix to say *undetected*. *Provenance recheck* recomputes every input artefact's SHA-256 against the hashes recorded in a result bundle.

Datasets integrated. Six integrations exist at the time of writing: the HUST bearing archive (60 files) [4], a triaxial bearing dataset (36 files) [10], a two-stage gearbox archive (20 files, healthy vs. broken tooth across ten loads) [13], the acoustic (five files) and vibration (45 files) subsets of a multimodal rotating-machine dataset [5], and one synthetic example plugin used in CI. All raw data remain unredistributed under CC BY 4.0; registries pin byte sizes and SHA-256 values per file, and archive hashes are verified against the values published by the sources where available. Source quirks are handled explicitly rather than silently: an unstated sampling rate becomes `None` with fail-closed consumers, and an upstream filename misspelling is normalized for labels while the original filename stays pinned for hashing.

Statistics and CI.. The statistics module implements group bootstrap and paired comparisons over recording or evaluation-group units. Continuous integration runs the test suite and lint on Python 3.10/3.12 (Ubuntu), schema-validates every registered artefact, executes an independently packaged plugin end to end, re-runs the fault-injection matrix, model-checks the TLA$^+$ specification on a Java runner, and—for reproduction evidence—executes the same frozen task on Ubuntu and macOS runners and fails if the canonical per-seed metrics differ.

6 EVALUATION

The evaluation answers four research questions. **RQ1**: do the formal constraints guarantee experiment integrity, and does the verification detect injected violations? **RQ2**: does the plugin architecture support independent extension without kernel changes? **RQ3**: are experiments traceable and reproducible? **RQ4**: does the framework sustain heterogeneous data and acceptable overhead? Unless stated otherwise, all numbers come from artefacts committed with the repository (TLC log, detection-matrix JSON, result bundles, benchmark scripts).

6.1 RQ1: Integrity verification

Model checking. TLC explores the lifecycle model exhaustively over the configured finite constants and reports no errors: 89,476 states generated, 46,686 distinct states, depth 11, no deadlocks, all nine properties (type safety, INV-1 through INV-6, fail-closed discipline) holding in every reachable state. Every rejection path (tampered partitions, foreign-partition windows, test-scope fits, contract-less plugins, incomplete bundles) is exercised by construction, because the spec offers the bad candidate non-deterministically and TLC explores it.

Property-based testing. Eight oracle properties run in the default suite. Each pairs a generated input with an *independently recomputed* expected outcome—for example, the recording audit must pass if and only if the generated set has unique identifiers, supported labels, and fully assigned conditions—so agreement with the oracle is asserted, not merely acceptance of valid cases. The properties cover task-partition validation, recording-audit agreement, window-partition totality and exclusivity under relabelling, prepared-table round trips, window-identity uniqueness, seed bijection, training-policy validation, and plugin-contract rejection.

Fault injection. Table 2 shows the twelve-entry catalogue: every planted violation is rejected fail-closed by at least one mechanism, with the violated invariant named in the error. Nine distinct mechanisms participate. The matrix reporter itself is tested for honesty: a canary test disables one guard and requires the report to say *undetected* rather than silently passing.

The enforcement is not decorative: during the gearbox integration, the length-consistency check at stream open rejected the first real extraction run because the registry's row counts—naively counted—overstated each file by two trailing blank lines. The registry was corrected before any result existed, and the incident is preserved in the integration decision record.

Table 2. Fault-injection detection matrix (all entries executed against the live kernel).

Mutation	Target	Detecting mechanism
M-LEAK-01	INV-2	pre-training recording audit
M-LEAK-02	INV-2	recording cross-audit vs. canonical metadata
M-LEAK-03	INV-3	kernel fit-scope guard (state digest differs from all-rows fit)
M-LEAK-04	INV-3	partition-restricted views (fit never sees test rows)
M-HASH-01	INV-1	adapter checksum verification at open
M-SEED-01	INV-4	result validator (bijection)
M-SEED-02	INV-4	result validator (duplicate seed)
M-PROV-01	INV-5	result validator (required fields)
M-PROV-02	INV-5	input-artifact hash recheck after tampering
M-PLUGIN-01	INV-6	plugin registry identity check
M-PLUGIN-02	INV-6	plugin contract validation
M-PLUGIN-03	INV-6	plugin registry collision rejection

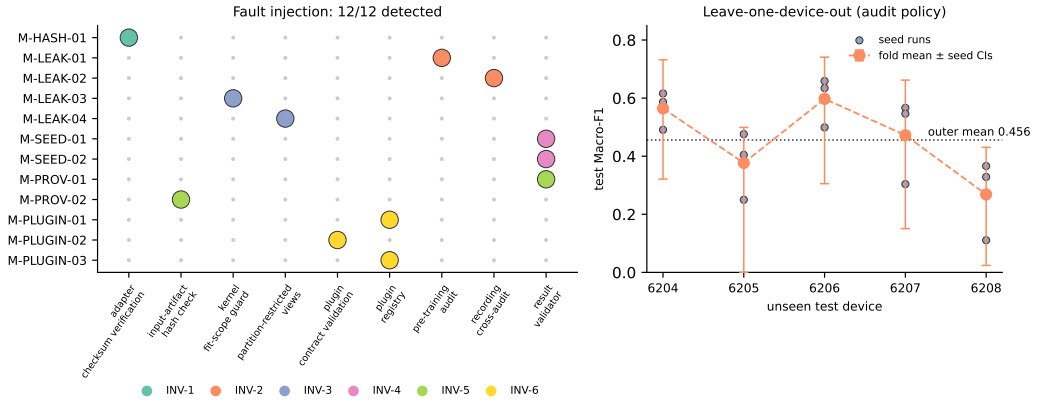


Fig. 3. Verification and evaluation evidence from committed artefacts. Left: the fault-injection catalogue—every one of the twelve planted violations (grouped by target invariant) is detected fail-closed by at least one of the nine kernel mechanisms. Right: leave-one-device-out folds on the bearing archive (audit policy): per-seed test Macro-F1 points, fold means with bars spanning the per-seed recording-bootstrap 95% CIs (exported as committed bootstrap files), and the outer mean of 0.456; device identity, not load shift, dominates the failure mode.

6.2 RQ2: Independent extension

Table 3 quantifies the two most recent integrations. All adapter-side code, tests, registries, task manifests, and documentation sum to 812 and 595 lines respectively; the lifecycle kernel gained no dataset-specific logic. A committed test enforces this structurally: no kernel module may contain a dataset identifier, with the two known debts (the fetch dispatcher and one frozen demonstration asset) declared in the test itself. Kernel touches that did occur were generic (an optional sampling-rate field for sources that state none) and are classified in the integration-effort audit.

Table 3. Integration effort for the two most recent datasets.

Integration	Adapter LOC	Test LOC	Files added (all artefacts)
Gearbox (20 recordings)	239	175	6
KAIST acoustic (5 recordings)	261	158	5

Compared with the script-per-dataset tradition, the operations that used to require editing experiment code—adding a dataset, adding a model, changing the held-out conditions, checking leakage, reproducing a result—now live in, respectively, an adapter package, a model plugin, a task manifest, an automated audit, and a result bundle with hash recheck. The boundary is candid: both audited integrations were performed by the author; soliciting one non-author integration is an open item, and the repository ships an issue template with an acceptance checklist for exactly that purpose.

6.3 RQ3: Traceability and reproduction

Traceability is content-addressed: every result bundle hashes its task manifest, feature source (table or dataset manifest), and training policy, and `results validate --check-hashes` recomputes them; tampering with an input after the fact is detected as a mismatch (M-PROV-02). The strongest reproduction evidence to date is same-commit, cross-Python: the development environment (3.11) and an independent virtual environment (3.12, package and external plugin installed from source) produce identical per-seed test Macro-F1 and worst-condition scores on a frozen task. A CI gate extends this across operating systems—the same task runs on Ubuntu and macOS runners and the comparison job fails on any divergence in canonical metrics; its first executed run accompanies the public push of this repository. Full multi-machine, multi-investigator reproduction remains future work.

6.4 RQ4: Heterogeneity, scale, and overhead

Heterogeneous tasks, executed on real data. Three task families newly enabled by the v1.2 architecture were executed on real archives end to end (streaming from checksum-verified sources, three frozen seeds each, group-bootstrap intervals over test recordings). Under a deliberately reduced two-epoch *audit policy*—chosen to validate mechanism, not to claim scores—the gearbox cross-load task yields window-level Macro-F1 of 0.333 with a recording-level 95% CI of $[0.11, 0.47]$ over eight test recordings; the combined-condition load×defect-size task yields 0.330–0.337 with CIs around $[0.14, 0.46]$ over six test recordings. The five-fold leave-one-device-out family on the bearing archive produces fold means of 0.565/0.377/0.597/0.472/0.268 (outer mean 0.456) with large per-seed variance—consistent with, and sharper than, the frozen v1.0 finding that device hold-out fails badly while load hold-out looks deceptively good. These runs demonstrate what the framework now makes routine—combined shifts, device rotation, and recording-level uncertainty on real, hash-verified archives—and are labelled non-protocol in the artefacts; the full-protocol v1.0/v1.1 matrices (3 tasks × 4 models and 2 tasks × 4 models, with negative results retained) were produced by earlier versions of the same pipeline and remain the benchmark claims of record.

Framework overhead. Table 4 measures the kernel's cost on synthetic tables. Loading plus schema-validating a 100,000-row v2 table costs 8.8 s against 0.57 s for bare JSON parsing (15.6×)—the price of constructing immutable, identity-carrying rows and validating them; the optional Parquet backend addresses exactly this cost for large matrices. Result validation is 0.7 ms per bundle. Streaming extraction plus audit of the 36-file triaxial archive (1,154 windows) completes in about 1.5 s.

Table 4. Kernel load-and-validate cost versus bare `json.loads` (median of five repeats).

Rows	Kernel (s)	Bare JSON (s)	Ratio
1,000	0.086	0.0028	30.1
10,000	0.780	0.0287	27.1
100,000	8.813	0.5667	15.6

Modality and format coverage. The signal layer presently streams four source formats (MAT v5, Siemens Test.Lab MAT, CSV, tab-separated text) across vibration, acoustic pressure, and gearbox accelerometers, with per-file rates from 25.6 kHz to 51.2 kHz and one archive whose rate is honestly None. One archive (the multimodal vibration set, 45 files) is integrated and audit-verified but its full end-to-end run awaits a machine with more memory than the development host (8 GB); the data card records this boundary.

6.5 Threats to validity

Internal. The fault catalogue plants artefact-level violations by hand; it is not exhaustive source-code mutation, and a guard outside the catalogue's reach is untested until a property or real incident exercises it. *External.* Verification uses many synthetic fixtures; real-archive coverage exists for four of six integrations, and the audit-policy training runs are not comparable to the frozen protocol results. *Construct.* "Formally specified" covers the abstract lifecycle; implementation claims rest on tests, runtime checks, and injection results, and the paper keeps the two layers separate. *Adoption.* All integrations and audits to date are author-performed; third-party evidence is solicited but absent.

7 DISCUSSION

What the approach buys. The value of pushing integrity into architecture rather than documentation shows up in three places. Extension becomes safe by default: a new dataset or model arrives as data plus a contract-checked adapter, and the invariants do not need to be re-explained to it because it cannot reach around them. Verification becomes possible at all: an invariant that exists as a guard in a state machine can be model-checked, generated against, and attacked with injected faults, which a convention in a script cannot. And results become contestable in a productive way: because bundles hash their inputs and retain predictions, a sceptical reader can recompute rather than re-trust.

Where the boundary of the guarantee lies. The formal layer certifies the design's guard logic, not the Python program; a bug in a guard's implementation is caught only by the implementation-level layers (property tests, injection, real incidents such as the registry row-count catch). The catalogue covers twelve violation families—the ones the invariants name—and honest reporting means admitting that a thirteenth kind of fault, if conceived, must first be added to the catalogue before its detection can be claimed. Likewise, test isolation is structural with respect to the framework's APIs; a plugin that exfiltrates test data through side channels outside the kernel's view is outside this threat model, as is malicious third-party code at install time (a standing, documented caveat of entry-point discovery).

Limitations. The framework has one author; every integration, audit, and reproduction to date is author-performed, and the strongest missing evidence is a third-party integration with recorded friction. Cross-environment reproduction currently spans Python versions and (via CI) operating systems, not physical machines and investigators. One integrated archive awaits a larger machine for

its full run; the temperature/current modality needs a TDMS reader. The statistical layer addresses non-independence across recordings, but deeper dependence structures (shared test rigs, shared campaigns) are dataset knowledge the framework can express only through user-declared grouping fields. Finally, the reduced-policy demonstration runs must never be quoted as benchmark scores; the artefacts carry that warning on every file.

Future work. Near-term: execute the vibration task on suitable hardware; pluginize the fetch dispatcher (the last dataset-identifying kernel debt); add the temperature/current modality; and run the full RQ1–RQ4 evidence matrix at protocol scale, including cross-machine reproduction. Medium-term: streaming windowing to remove the memory ceiling, seeding the statistics layer from bundle-retained predictions for automatic CI checks of published results, and exploring Alloy-style small-scope checks of the partition structures themselves.

8 CONCLUSION

EdgeFault-Bench reframes the credibility problem of cross-condition fault diagnosis as a software-engineering problem and answers it the way software engineering answers such questions: with a specification, an architecture that enforces it, and evidence that the enforcement works. The experiment lifecycle is a guarded state machine whose six integrity invariants are model-checked clean over all reachable states, exercised by oracle-based property tests, and attacked by a fault-injection catalogue that every guard survives fail-closed. Heterogeneous datasets—bearings, a gearbox, acoustic and vibration archives with formats, rates, and quirks that include an unstated sampling rate and an upstream misspelling—integrate as plugins while the kernel stays free of dataset logic, asserted by test. Results carry content-hashed provenance, complete seed bookkeeping with failures retained, and recording-level bootstrap uncertainty; reproduction holds exactly across Python versions and is gated across operating systems in CI. The costs are measured, and the boundaries are stated where evidence ends: the formal layer certifies the design, third-party adoption evidence is solicited rather than claimed, and non-protocol demonstration runs are labelled as such. We believe the wider lesson generalizes beyond fault diagnosis: when an empirical field's conclusions are fragile, the benchmark infrastructure itself deserves specification, enforcement, and adversarial verification—not just standardization of datasets and metrics.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE MANUSCRIPT PREPARATION PROCESS

During the preparation of this work the author used the GLM large-language model (as the coding and writing assistant of the ZCode environment) to support implementation of the framework, execution of the verification workflows, and drafting of manuscript text. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

DECLARATION OF COMPETING INTEREST

The author declares no competing interests.

FUNDING

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

DATA AVAILABILITY

The framework, formal models, verification artefacts (TLC log, fault-injection matrices, benchmark scripts), frozen task registries, and all result bundles referenced in this article are available in the project's public repository. To preserve double-anonymized review, the repository link is provided to the editors through the submission system and will be cited by its persistent identifier in the published version. Underlying datasets are third-party CC BY 4.0 archives that are not redistributed; each is pinned by checksums in the repository and cited in the reference list.

REFERENCES

[1] Mohammed M. Farag et al. 2025. Towards a Standard Benchmarking Framework for Domain Adaptation in Intelligent Fault Diagnosis. *IEEE Access* 13 (2025), 24426–24453. doi:10.1109/ACCESS.2025.3537817.

[2] Sasi Kiran Gaddipati, D. Muhammed, F. Keya, et al. 2026. MLReplicate: Benchmarking Autonomous Research Systems for Machine Learning Reproducibility. arXiv:2605.16616.

[3] Ishaan Gulrajani and David Lopez-Paz. 2021. In Search of Lost Domain Generalization. In *International Conference on Learning Representations (ICLR)*. arXiv:2007.01434.

[4] Hoang Si Hong and Nguyen Thuan. 2023. HUST bearing: a practical dataset for ball bearing fault diagnosis. Mendeley Data, v3. doi:10.17632/cbv7jyx4p9.3.

[5] Wonho Jung, Seong-Hu Kim, SungHyun Yun, Jaewoong Bae, and Yong-Hwa Park. 2023. Vibration, acoustic, temperature, and motor current dataset of rotating machine under varying load conditions for fault diagnosis. Mendeley Data, v6. doi:10.17632/ztmf3m7h5x.6.

[6] Julian Kramer and Tianxiang Lu. 2025. A Reproducible Framework for Benchmarking Machine Learning Operations (MLOps) Infrastructures: Comparing Bare-Metal and Orchestrated Machine Learning Workflows. *Cureus* (2025).

[7] Leslie Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

[8] Bohui Lyu. 2026. When Code Does Not Run: Reproducibility Challenges in Materials Machine Learning Benchmarks. *AI Agent* 2, 3 (2026), 23. doi:10.20517/aiagent.2026.31.

[9] David R. MacIver et al. [n. d.]. Hypothesis: property-based testing for Python. https://hypothesis.readthedocs.io. Accessed November 2026.

[10] Mehran University of Engineering and Technology / NCRA. [n. d.]. Triaxial bearing vibration dataset of induction motor under varying load conditions. Mendeley Data, v2. doi:10.17632/fm6xzxnf36.2.

[11] Matthew Middlehurst et al. 2024. aeon: a Python toolkit for learning from time series. *Journal of Machine Learning Research* 25, 289 (2024), 1–10.

[12] MLflow contributors. 2018. MLflow: An open source platform for the machine learning lifecycle. https://mlflow.org. Accessed November 2026.

[13] Yogesh Pandya. [n. d.]. Gearbox fault diagnosis data (two-stage gearbox). U.S. Department of Energy / Data.gov. https://catalog.data.gov/dataset/gearbox-fault-diagnosis-data, CC BY 4.0.

[14] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*.

[15] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[16] Chao Zhao. 2024. Domain-generalization fault-diagnosis benchmark. https://github.com/CHAOZHAO-1/Domain-generalization-fault-diagnosis-benchmark. Code accompanying [17].

[17] Chao Zhao, Enrico Zio, and Weiming Shen. 2024. Domain generalization for cross-domain fault diagnosis: An application-oriented perspective and a benchmark study. *Reliability Engineering & System Safety* 245 (2024), 109964. doi:10.1016/j.ress.2024.109964.